

A decorative graphic consisting of a thin gold circle on the left side. A horizontal bar, divided into a gold left half and a light gray right half, passes through the center of the circle. Large black and gold brackets are positioned on the left and right sides of the bar, respectively.

Langage de Programmation C

Département de la formation préparatoire ST
Ecole Nationale Polytechnique d'Oran

[Partie 4]

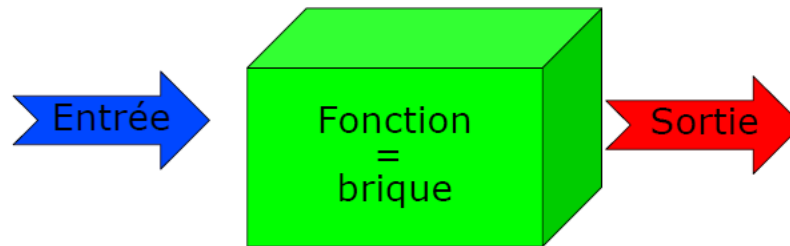
■ Fonctions

Introduction programmation modulaire

- Jusqu'ici nous sommes restés à l'intérieur de la fonction **main** (entre ses deux accolades). Nous n'en sommes jamais sortis parce que nos programmes étaient **courts**.
- Certains problèmes conduisent à des programmes longs, difficiles à écrire et à comprendre. On les découpe en des parties appelées **sous-programmes** ou **modules**.
- La structuration de programmes en sous-programmes se fait en **C** à l'aide des **fonctions**

Introduction programmation modulaire

- Une fonction exécute des actions et renvoie un résultat. C'est un **morceau de code** qui sert à faire quelque chose de précis.
- On dit qu'une fonction possède une entrée et une sortie. La fig. suivante représente une fonction schématiquement.



[Introduction programmation modulaire]

- Les fonctions:
 - permettent de "**factoriser**" les programmes, càd de mettre en commun les parties qui se répètent
 - permettent une **structuration** et une **meilleure lisibilité** des programmes
 - **facilitent la maintenance** du code (il suffit de modifier une seule fois)
 - peuvent éventuellement être **réutilisées** dans d'autres programmes

programmation modulaire (Fonctions)

- En général, le nom d'une fonction apparaît à trois endroits dans un programme:

- 1) lors de la **déclaration**

- 2) lors de la **définition**

- 3) lors de **l'appel**

```
#include <stdio.h>
```

```
/* Le prototype */
```

```
int main()
```

```
{
```

```
...
```

```
/* L'appel */
```

```
...
```

```
}
```

```
/* La définition */
```

[La programmation modulaire exemple]

Exemple

Dans le programme suivant, la fonction **main** utilise les deux fonctions:

- **ENTREE** qui lit un nombre entier au clavier et le fournit comme résultat. La fonction ENTREE n'a pas de paramètres.
- **MAX** qui renvoie comme résultat le maximum de deux entiers fournis comme paramètres.

```

main()
{
    /* Prototypes des fonctions appelées */
    int ENTREE(void);
    int MAX(int N1, int N2);
    /* Déclaration des variables */
    int A, B;
    /* Traitement avec appel des fonctions */
    A = ENTREE();
    B = ENTREE();
    printf("Le maximum est %d\n", MAX(A,B));
}

/* Définition de la fonction ENTREE */
int ENTREE(void)
{
    int NOMBRE;
    printf("Entrez un nombre entier : ");
    scanf("%d", &NOMBRE);
    return NOMBRE;
}

/* Définition de la fonction MAX */
int MAX(int N1, int N2)
{
    if (N1>N2)
        return N1;
    else
        return N2;
}

```

1. Le prototypage

2. La définition

3. L'appel

Définition de fonctions

- Une fonction est définie en *dehors de la fonction principale main () en indiquant :*
 - le nom de la fonction
 - le type, le nombre et les noms des paramètres de la fonction
 - le type du résultat fourni par la fonction
 - les données locales à la fonction
 - les instructions à exécuter
- **Attention !**
- Si nous choisissons un nom de fonction qui existe déjà dans une bibliothèque, notre fonction **cache** la fonction prédéfinie.

Type du résultat fourni par la fonction

- Une fonction peut fournir comme résultat:
 - *un type arithmétique(int, float, char),*
 - une structure (définie par **struct** - non traité dans ce cours),
 - une réunion (définie par **union** - non traité dans ce cours),
 - un pointeur
 - **void** (la fonction correspond alors à une 'procédure').

Définition de fonctions:

Fonction avec return

■ 1. Fonction avec return

c'est un sous-programme permettant de calculer et de "retourner" (avec **return**) un seul résultat de type simple (float, char et int dans le cadre du cours) à partir d'une liste de paramètres (des données).

■ Syntaxe:

arguments de la fonction et leurs types.

type du résultat retourné par la fonction. → type **nom_fonction** (type1 arg1,..., typeN argN) en-tête de la fonction

```
{  
    instructions constituant le corps de la fonction  
    return (expression)  
}
```

Remarque:

Si une fonction n'a pas de paramètres, on peut déclarer la liste des paramètres comme **(void)** ou simplement comme **()**

Définition de fonction avec « return » exemples:

Exemple1:

Une fonction qui calcule la somme de deux réels x et y :

```
double Som(double x, double y)
{
    return (x+y);
}
```

Exemple3 : Une fonction qui retourne le plus grand entier:

```
float plusGrand(float x, float y)
{
    if (x > y)
        return x;
    else
        return y;
}
```

Exemple2:

fonction qui calcul le Δ .

```
double CalcDisc(double Arga, double Argb, double Argc)
{
    return Argb*Argb-4*Arga*Argc;
}
```

Exemple 4: Une fonction qui renvoie un entier saisi au clavier:

```
int entree(void)
{
    int n;
    printf (" Entrez n \n");
    scanf ("%d ", &n);
    return n;
}
```

Remarques

- L'instruction

return <expression>;

a les effets suivants:

- *évaluation de l'<expression>*
- *conversion automatique du résultat de l'expression dans le type de la fonction*
- *renvoi du résultat*
- *termination de la fonction*

- L'instruction "return" provoque la fin de la fonction. **On revient ensuite à l'endroit où la fonction a été appelée.** Il doit avoir au moins une instruction "return" dans la fonction.

- Il faut utiliser le résultat retourné dans un bon contexte (affichage, affectation, comparaison, ...).

- Dans l'en-tête, on ne peut pas regrouper les paramètres de même type :

float plusGrand(float x, y) /* erroné! */

- Ne pas terminer l'en-tête par un point virgule :

float plusGrand(float x, float y); /* erroné! */

Définition de fonctions:

Fonction de type void

■ 2. Fonction de type void (pas de return)

Ce genre de fonction est semblable au Procedure en PASCAL. **Elle réalise une ou quelques tâches** (trier, chercher, compter-afficher, calculer, etc.).

■ Syntaxe:

```
void nom_fonction (type1 arg1,..., typeN argN)
```

```
{
```

Suite d'instructions constituant le corps de la fonction
réalisant la tâche confiée à la fonction

```
}
```

■ Remarque

Pour le nom de la fonction (nom_fonction), il est plus simple d'utiliser le nom d'un verbe (ou venant d'un verbe) qui résume la tâche de la fonction :

- void calculer(...)
- void trier(...)
- void lireRemplir(...)
- etc.

Fonction de type « void » exemples:

Exemple5: fonction qui affiche les éléments d'un tableau d'entiers

```
void AfficheTableau(float *ptab, int dim)
{
    int i;
    for (i=0; i<dim ; i++)
        printf("%f", *(ptab+i) );
}
```

```
void AfficheTab( int T[ ], int dim)
{
    int i;
    for(i=0;i<dim;i++)
        printf (" %d \t", T[i]);
}
```

Exemple6: Une fonction qui affiche la somme de deux réels x et y :

```
void AfficheSom(double x, double y)
{
    printf (" %lf", x+y );
}
```

[Remarques]

- Le type par défaut est **int**; autrement dit: si le type d'une fonction n'est pas déclaré explicitement, elle est automatiquement du type **int**.
- Il est interdit de définir des fonctions à l'intérieur d'une autre fonction.
- En principe, l'ordre des définitions dans le texte du programme ne joue pas de rôle, mais chaque fonction doit être **déclarée ou définie avant d'être appelée**.

Appel d'une fonction

- L'appel d'une fonction se fait par simple écriture de son nom avec la liste des paramètres : **nom_fonction (para1,..., paraN)**

Exemple: Appel de la fonction

```
main()  
{ double z;  
  int A[5] = {1, 2, 3, 4, 5};  
  z=Som(2.5, 7.3);  
  AfficheTab(A,5);  
}
```

paramètres effectifs

Définition de la fonction: paramètres formels.

```
double Som(double x, double y )  
{  
    return (x+y);  
}
```

- Lors de l'appel d'une fonction, les paramètres sont appelés **paramètres effectifs** : ils contiennent les valeurs pour effectuer le traitement.
- Lors de la définition, les paramètres sont appelés **paramètres formels**.
- L'ordre et les types des paramètres effectifs doivent correspondre à ceux des paramètres formels

[Appel d'une fonction]

- *Les paramètres d'une fonction sont simplement des **variables locales** qui sont initialisées par les **valeurs** obtenues lors de l'appel.*

Déclaration des fonctions

- Il est nécessaire pour le compilateur de connaître la définition d'une fonction au moment où elle est appelée. Si une fonction est définie après son premier appel (en particulier si elle est définie après main), elle doit être **déclarée** auparavant.
- La déclaration d'une fonction se fait par son **prototype** qui indique les types de ses paramètres et celui de la fonction :

`type nom_fonction (type1,..., typeN)`

- Il est interdit en C de définir des fonctions à l'intérieur d'autres fonctions. En particulier, on doit définir les fonctions soit avant, soit après la fonction principale main.
- **Exemple**
`void AfficheTableau(float *ptab, int nlignes);`

Etude d'un Exemple

Soient les fonctions **main**, **PI** et **SURFACE** définies par:

```
#include <stdio.h>

main()
{
    double R;
    printf("Introduire le rayon du cercle : ");
    scanf("%lf", &R);
    printf("La surface du cercle est %f. \n", SURFACE(R));
    return 0;
}

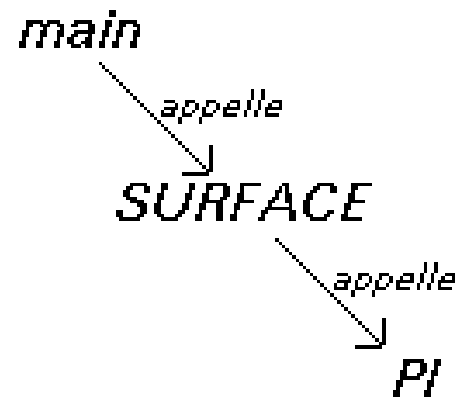
double PI(void)
{
    return 3.14159265;
}

double SURFACE(double RAYON)
{
    return PI()*RAYON*RAYON;
}
```

- a) Etablir la hiérarchie des appels pour les trois fonctions.
- b) Corriger le programme.

Etude d'un Exemple

a) Etablir la hiérarchie des appels pour les trois fonctions:



Etude d'un Exemple

- Déclarations locales des fonctions et définition 'top-down':

```
#include <stdio.h>

main()
{
    /* Déclaration locale de SURFACE */
    double SURFACE(double RAYON);
    double R;
    printf("Introduire le rayon du cercle : ");
    scanf("%lf", &R);
    printf("La surface du cercle est %f. \n", SURFACE(R));
    return 0;
}

double SURFACE(double RAYON)
{
    /* Déclaration locale de PI */
    double PI(void);
    return PI()*RAYON*RAYON;
}

double PI(void)
{
    return 3.14159265;
}
```

Etude d'un Exemple

Définition 'bottom-up' sans déclarations:

```
#include <stdio.h>

double PI(void)
{
    return 3.14159265;
}

double SURFACE(double RAYON)
{
    return PI() * RAYON * RAYON;
}

main()
{
    double R;
    printf("Introduire le rayon du cercle : ");
    scanf("%lf", &R);
    printf("La surface du cercle est %f. \n", SURFACE(R));
    return 0;
}
```

Etude d'un Exemple

- Déclaration globale des fonctions et définition 'top-down':

```
#include <stdio.h>
/* Déclaration globale des fonctions */
double SURFACE(double RAYON);
double PI(void);

main()
{
    double R;
    printf("Introduire le rayon du cercle : ");
    scanf("%lf", &R);
    printf("La surface du cercle est %f. \n", SURFACE(R));
    return 0;
}

double SURFACE(double RAYON)
{
    return PI() * RAYON * RAYON;
}

double PI(void)
{
    return 3.14159265;
}
```


La notion de blocs et la portée des identificateurs

- Les fonctions en C sont définies à l'aide de blocs d'instructions. Un bloc d'instructions est encadré d'accolades et composé de deux parties:
- ***Blocs d'instructions en C***

```
{    <déclarations locales>  
    <instructions>  
}
```

La notion de blocs et la portée des variables

- Deux types de variables sont manipulés dans un programme C :
 - **variables locales**
 - **variables globales.**

Elles se distinguent par ce qu'on appelle leur **portée** (leur "espace de visibilité", leur "durée de vie")

1. Variables locales

Les variables déclarées dans un bloc d'instructions (particulièrement une fonction) sont *uniquement visibles à l'intérieur de ce bloc*. On dit que ce sont des **variables locales** à ce bloc(fonction).

Elles sont créées à l'appel de la fonction et détruites à la fin de son exécution

Attention !

- Une variable déclarée à l'intérieur d'un bloc **cache** toutes les variables du même nom des blocs qui l'entourent.

La notion de blocs et la portée des variables

2. Variables globales

- Les variables déclarées au **début du fichier, à l'extérieur** de toutes les fonctions *sont disponibles à toutes les fonctions du programme*. Ce sont alors des **variables globales**.
- En général, les variables globales sont déclarées immédiatement derrière les instructions **#include** au début du programme.

Attention !

*Les variables déclarées au début de la fonction principale **main** **ne sont pas** des variables globales, mais elles sont locales à **main** !*

Variables locales et globales exemple

```
#include<stdio.h>
int x = 7;
int f(int);
int g(int);
main( )
{
    printf("x = %d\t", x);
    {
        int x = 6;    printf("x = %d\t", x); }
    printf("f(%d) = %d\t", x, f(x));
    printf("g(%d) = %d\t", x, g(x));
}
int f(int a) { int x = 9;    return (a + x); }
int g(int a) { return (a * x); }
```

Qu'affiche ce programme?

x=7 x=6 f(7)=16 g(7) = 49

Variables locales et globales exemple

```
#include<stdio.h>
void f(void);
int i;
main( )
{ int k = 5;
i=3; f(); f();
printf("i = %d et k=%d \n", i,k); }
void f(void) { int k = 1;
printf("i = %d et k=%d \n", i,k);
i++;k++;}
```

Qu'affiche ce programme?

i=3 et k=1

i=4 et k=1

i=5 et k=5

Remarques

- Les variables globales sont à utiliser avec précaution, puisqu'elles créent des *liens invisibles entre les fonctions*.
- La modularité d'un programme peut en souffrir et le programmeur risque de perdre la vue d'ensemble.
- Il faut faire attention à ne pas cacher involontairement des variables globales par des variables locales du même nom.
- Il est conseillé ***d'écrire nos programmes aussi 'localement' que possible***.
- ***L'utilisation de variables globales devient inévitable***, si plusieurs fonctions qui ne s'appellent pas ont besoin des mêmes variables ou plusieurs fonctions d'un programme ont besoin du même ensemble de variables. Ce serait alors trop encombrant de passer toutes les variables comme paramètres d'une fonction à l'autre.

[Paramètres d'une fonction]

- Les *paramètres* ou *arguments* sont les 'boîtes aux lettres' d'une fonction. Ils servent à échanger des informations entre la fonction appelante et la fonction appelée. Ils peuvent recevoir des données et stocker des résultats
- ***Conversion automatique***
Lors d'un appel, le nombre et l'ordre des paramètres doivent nécessairement correspondre aux indications de la déclaration de la fonction. Les paramètres sont automatiquement convertis dans les types de la déclaration avant d'être passés à la fonction.

Passage de paramètres d'une fonction

Il existe deux modes de transmission de paramètres dans les langages de programmation :

1. Passage des paramètres par valeur

- les valeurs des paramètres *effectifs* sont affectées aux paramètres *formels* correspondants au moment de l'appel de la fonction ou procédure.
- c.-à-d. les fonctions n'obtiennent que les *valeurs* de leurs paramètres et n'ont pas d'accès aux variables elles-mêmes.
- *Dans ce mode le paramètre effectif ne subit aucune modification*

Passage de paramètres d'une fonction

2. Passage des paramètres par adresse (ou par référence)

- les adresses des paramètres effectifs sont transmises à la fonction appelante.
- *Dans ce mode, le paramètre effectif subit les mêmes modifications que le paramètre formel*
- **Remarque**
- En C, le passage des paramètres se fait toujours par la valeur,
- *Pour changer la valeur d'une variable de la fonction appelante, nous allons procéder comme suit:*
 - La fonction appelante doit **fournir l'adresse de la variable** et
 - La fonction appelée doit **déclarer le paramètre comme pointeur**.
 - On peut alors atteindre la variable à l'aide du pointeur.

[Discussion d'un exemple]

Ecrire la fonction **PERMUTER** qui échange le contenu de deux variables du type **int**

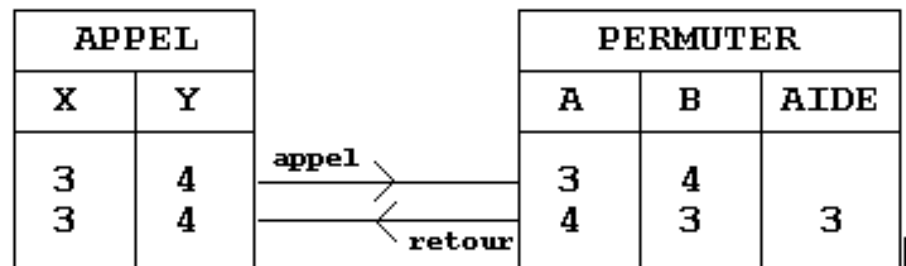
- En première approche, nous écrivons la fonction suivante:

```
void PERMUTER (int A, int B)
{ int AIDE;
  AIDE = A;
  A = B;
  B = AIDE;
}
```

- Nous appelons la fonction pour deux variables X et Y par:

PERMUTER(X, Y); → **Résultat:** X et Y restent **inchangés** !

Explication: Lors de l'appel, les *valeurs* de X et de Y sont copiées dans les paramètres A et B. **PERMUTER** échange bien contenu des variables *locales* A et B, mais les valeurs de X et Y restent les mêmes.



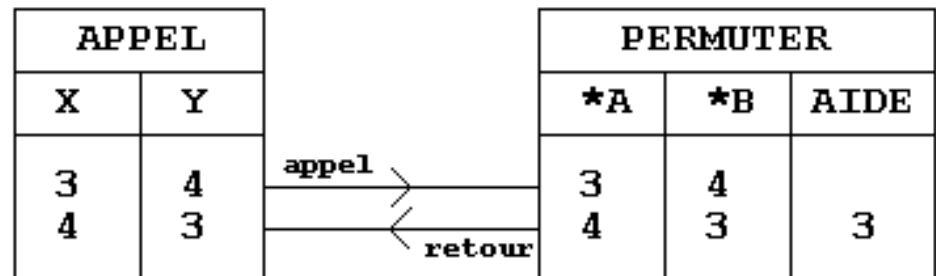
Discussion d'un exemple

- Pour pouvoir modifier le contenu de X et de Y, la fonction PERMUTER a besoin des *adresses* de X et Y.
- En utilisant des pointeurs, nous écrivons une deuxième fonction:

```
void PERMUTER (int *A, int *B)
{ int AIDE;
  AIDE = *A;
  *A = *B;
  *B = AIDE;}
```

- Nous appelons la fonction par: **PERMUTER(&X, &Y);**
- **Résultat:** → Le contenu des variables X et Y est échangé !

Explication: Lors de l'appel, les *adresses* de X et de Y sont copiées dans les *pointeurs* A et B. **PERMUTER** échange ensuite le contenu des adresses indiquées par les pointeurs A et B.



[On resume]

- En C, le passage des paramètres se fait toujours par la valeur,
- Pour *changer* la *valeur* d'une variable de la fonction *appelante*, nous allons procéder comme suit:
 - La fonction appelante doit **fournir l'adresse de la variable** et
 - La fonction appelée doit **déclarer le paramètre comme pointeur**.
 - On peut alors atteindre la variable à l'aide du pointeur.

Passage de l'adresse d'un tableau à une dimension

Passage de l'adresse d'un tableau à une dimension

■ **Méthode**

- Comme il est impossible de passer 'la valeur' de tout un tableau à une fonction, on fournit ***l'adresse d'un élément du tableau***.
- En général, on fournit l'adresse du premier élément du tableau, qui est donnée par ***le nom du tableau***.

■ **Déclaration**

- Dans la liste des paramètres d'une fonction, on peut déclarer un tableau par le nom suivi de crochets, **<type> <nom>[]**
- ou simplement par un pointeur sur le type des éléments du tableau: **<type> *<nom>**

■ **Appel**

- Lors d'un appel, l'adresse d'un tableau peut être donnée par le nom du tableau, par un pointeur ou par l'adresse d'un élément quelconque du tableau.

Exemple d'application

1. La fonction LIRE_TAB à trois paramètres TAB, N et NMAX lit la dimension N et les composantes d'un tableau TAB du type **int**. La dimension N doit être inférieure à NMAX. Implémenter la fonction LIRE_TAB en choisissant bien le type des paramètres.

Exemple:

Pour un appel par

LIRE_TAB(T, &N, 10);

la fonction se comportera comme suit:

Dimension du tableau (max.10): 11

Dimension du tableau (max.10): 4

Élément[0] : 43

Élément[1] : 55

Élément[2] : 67

Élément[3] : 79

Exemple d'application

```
void LIRE_TAB (int TAB[], int *N, int NMAX)
{
    /* Variables locales */
    int I;
    /* Saisie de la dimension du tableau */
    do
    {
        printf("Dimension du tableau (max.%d) : ", NMAX);
        scanf("%d", N); /* Attention: écrire N et non &N ! */
    }
    while (*N<0 || *N>NMAX);
    /* Saisie des composantes du tableau */
    for (I=0; I<*N; I++)
    {
        printf("Elément[%d] : ", I);
        scanf("%d", &TAB[I]);
    }
}
```

Exemple d'application

```
void LIRE_TAB (int *TAB, int *N, int NMAX)
{
    /* Variables locales */
    int I;
    /* Saisie de la dimension du tableau */
    do
    {
        printf("Dimension du tableau (max.%d) : ", NMAX);
        scanf("%d", N); /* Attention: écrire N et non &N ! */
    }
    while (*N<0 || *N>NMAX);
    /* Saisie des composantes du tableau */
    for (I=0; I<*N; I++)
    {
        printf("Élément[%d] : ", I);
        scanf("%d", TAB+I);
    }
}
```


Exemple d'application

2. Ecrire la fonction ECRIRE_TAB à deux paramètres TAB et N qui affiche N composantes du tableau TAB du type **int**.

```
void ECRIRE_TAB (int *TAB, int N)
{
    /* Affichage des composantes du tableau */
    while (N)
    {
        printf("%d ", *TAB);
        TAB++;
        N--;
    }
    printf("\n");
}
```

Exemple d'application

3. Ecrire la fonction `SOMME_TAB` qui calcule la somme des `N` éléments d'un tableau `TAB` du type `int`. `N` et `TAB` sont fournis comme paramètres; la somme est retournée comme résultat du type `long`.

```
long SOMME_TAB(int *TAB, int N)
{
    /* Variables locales */
    long SOMME = 0;
    /* Calcul de la somme */
    while(N)
    {
        SOMME += *TAB;
        TAB++;
        N--;
    }
    return SOMME;
}
```

Exemple d'application

4. A l'aide des fonctions précédentes, écrire un programme qui lit un tableau A d'une dimension inférieure ou égale à 100 et affiche le tableau et la somme des éléments du tableau.

```
#include <stdio.h>

main()
{
    /* Prototypes des fonctions appelées */
    void LIRE_TAB (int *TAB, int *N, int NMAX);
    void ECRIRE_TAB (int *TAB, int N);
    long SOMME_TAB(int *TAB, int N);
    /* Variables locales */
    int T[100]; /* Tableau d'entiers */
    int DIM;    /* Dimension du tableau */
    /* Traitements */
    LIRE_TAB (T, &DIM, 100);
    printf("Tableau donné : \n");
    ECRIRE_TAB (T, DIM);
    printf("Somme des éléments du tableau : %ld\n", SOMME_TAB(T, DIM) );

    getch();
}
```

Fonctions récursives

- Une fonction qui fait appel à elle-même est une fonction récursive
- Toute fonction récursive doit posséder un cas limite (cas trivial) qui arrête la récursivité

- Exemple : Calcul du factorielle

```
int fact (int n )  
    { if (n==0) /*cas trivial*/  
      return (1);  
      else  
        return (n* fact(n-1) );  
    }
```

Remarque : l'ordre de calcul est l'ordre inverse de l'appel de la fonction

Fonctions récursives exemple

- Ecrivez une fonction récursive (puis itérative) qui calcule le terme n de la suite de Fibonacci définie par :

$$U(0)=U(1)=1$$

$$U(n)=U(n-1)+U(n-2)$$

```
int Fib (int n)
{
    if (n==0 || n==1)
        return (1);
    else
        return ( Fib(n-1)+Fib(n-2));
}
```

Fonctions récursives exemple(suite)

- Une fonction itérative pour le calcul de la suite de Fibonacci :

```
int Fib (int n)
{ int i, AvantDernier, Dernier, Nouveau;
  if (n==0 || n==1) return (1);
  AvantDernier=1; Dernier =1;
  for (i=2; i<=n; i++)
  { Nouveau= Dernier+ AvantDernier;
    AvantDernier = Dernier;
    Dernier = Nouveau;
  }
  return (Nouveau);
}
```

Remarque: la solution récursive est plus facile à écrire